

✓ Fine-Tuning an LLM for Sanskrit↔English (Option 1)

AI/ML Developer Assignment — ImmverseAI (BharatiyaGPT)

This notebook fine-tunes a small open-source instruction-tuned LLM using **QLoRA** to improve Sanskrit↔English translation, explanation, QA, and summarization.

Default / reported model: `Qwen/Qwen2.5-1.5B-Instruct` (ungated, no HF login needed). `meta-llama/Llama-3.2-1B-Instruct` is also supported via the same `BASE_MODEL` variable, but it's gated (needs a free HF account + license acceptance) and took ~18x longer to train under identical config in our testing — see `REPORT.md` section 3/8 for the full comparison.

Runtime: Google Colab, GPU = T4 or L4 (Runtime → Change runtime type → GPU)

Pipeline:

1. Setup & installs
2. Tokenizer inspection (Sanskrit vs English fragmentation)
3. Dataset construction (public parallel corpora → instruction pairs)
4. QLoRA fine-tuning
5. Inference (base vs fine-tuned)
6. Evaluation (BLEU / chrF++, before/after, failure analysis, ranked examples)
7. Save & export adapter

See `README.md` and `REPORT.md` in the repo for setup instructions and the full written report (problem understanding, tradeoffs, failure analysis, what we'd improve with more time).

✓ 1. Setup

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!unzip -o "/content/drive/MyDrive/module.zip" -d /content/
!ls -la /content
```

```
Archive: /content/drive/MyDrive/module.zip
  creating: /content/module/
  creating: /content/module/data/
  creating: /content/module/eval/
  inflating: /content/module/eval/evaluate.py
  creating: /content/module/outputs/
  creating: /content/module/scripts/
  inflating: /content/module/scripts/infer.py
  inflating: /content/module/scripts/inspect_tokenizer.py
  inflating: /content/module/scripts/prepare_data.py
  inflating: /content/module/scripts/train.py
total 24
drwxr-xr-x 1 root root 4096 Sep 13 11:47 .
drwxr-xr-x 1 root root 4096 Sep 13 11:41 ..
drwxr-xr-x 4 root root 4096 Sep  4 13:32 .config
drwx----- 5 root root 4096 Sep 13 11:45 drive
drwxrwxrwx 6 root root 4096 Sep 13 11:39 module
drwxr-xr-x 1 root root 4096 Sep  4 13:32 sample_data
```

```
!mv module/* module/.[!.*] . 2>/dev/null || true
!rm -rf module
```

```
# Clone the repo and flatten it into /content so later commands can use
# plain relative paths (scripts/..., eval/..., data/...).
!git clone https://github.com/yanil-03/sanskrit-llm.git
!mv sanskrit-llm/* sanskrit-llm/.[!.*] . 2>/dev/null || true
```

```
!rm -rf sanskrit-llm
```

```
# If you'd rather upload files manually instead of cloning: skip the cell
# above and just upload the `scripts/`, `eval/` folders alongside this
# notebook in Colab, then continue from here.
```

```
!pip install -q -U transformers accelerate peft trl bitsandbytes datasets sacrebleu sentencepiece
```

```
===== 12.3/12.3 MB 122.4 MB/s eta 0:00:00
===== 394.3/394.3 kB 34.9 MB/s eta 0:00:00
===== 1.0/1.0 MB 62.8 MB/s eta 0:00:00
===== 43.1/43.1 MB 24.7 MB/s eta 0:00:00
===== 559.1/559.1 kB 41.1 MB/s eta 0:00:00
===== 100.8/100.8 kB 8.9 MB/s eta 0:00:00
===== 129.0/129.0 kB 13.1 MB/s eta 0:00:00
```

```
import torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("VRAM (GB):", round(torch.cuda.get_device_properties(0).total_memory / 1e9, 1))
```

```
CUDA available: True
GPU: Tesla T4
VRAM (GB): 15.6
```

```
# Hugging Face auth — Llama models are gated; you need a free HF account
# and to accept the license at https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct
# then run:
from huggingface_hub import login
login() # paste your HF token when prompted
BASE_MODEL = "meta-llama/Llama-3.2-1B-Instruct"

# Default used to produce this repo's reported results: ungated, no login needed.
# BASE_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
BASE_MODEL
```



To log in, open this URL and enter the code:

<https://hf.co/oauth/device>

5 2 R 1 - D P Y E

Waiting for authorization...

Login successful. Logged in as **yanil-03** (token: oauth-yanil-03).
This token will be refreshed automatically when it expires.

```
'meta-llama/Llama-3.2-1B-Instruct'
```

2. Tokenizer Inspection

Before training, we check how the base tokenizer handles Devanagari/Sanskrit text compared to English of similar content. This is the "Tokenization / Vocabulary" challenge called out in the assignment — Sanskrit's compounding and rich morphology tend to fragment badly under tokenizers trained mostly on Latin-script/English corpora, which affects both training efficiency and the effective context window.

```
!python scripts/inspect_tokenizer.py --model "$BASE_MODEL"
```

```
config.json: 100% 877/877 [00:00<00:00, 2.86MB/s]
tokenizer_config.json: 100% 54.5k/54.5k [00:00<00:00, 12.3MB/s]
```



```
!python scripts/prepare_data.py --max_pairs 6000 --out_dir data
```

```
Loading rahular/itihasa via parquet (refs/convert/parquet) ...
Parquet load attempt failed (Unable to find 'hf://datasets/rahular/itihasa@~parquet/default/train/0000.parquet'); trying revis

Itihasa/train/0000.parquet: downloading bytes: 2% 334k/16.4M [00:01<01:21, 197kB/s]
Itihasa/train/0000.parquet: downloading bytes: 40% 6.52M/16.4M [00:01<00:01, 4.99MB/s, 32.0kB/s ]
Itihasa/train/0000.parquet: downloading bytes: 82% 13.5M/16.4M [00:01<00:00, 11.5MB/s, 623kB/s ]
Itihasa/train/0000.parquet: reconstructing file: 4% 699k/16.4M [00:02<00:59, 264kB/s, 43.9kB/s ]
Itihasa/train/0000.parquet: reconstructing file: 8% 1.34M/16.4M [00:03<00:22, 667kB/s, 61.6kB/s ]
Itihasa/train/0000.parquet: downloading bytes: 100% 16.3M/16.3M [00:03<00:00, 5.02MB/s, 1.38MB/s ]
Itihasa/train/0000.parquet: reconstructing file: 100% 16.4M/16.4M [00:03<00:00, 5.04MB/s, 1.50MB/s ]

Itihasa/validation/0000.parquet: downloading bytes: 0% 0.00/1.39M [00:00<?, ?B/s]
Itihasa/validation/0000.parquet: downloading bytes: 100% 1.39M/1.39M [00:01<00:00, 774kB/s, 134kB/s ]
Itihasa/validation/0000.parquet: reconstructing file: 100% 1.39M/1.39M [00:01<00:00, 777kB/s, 135kB/s ]

Itihasa/test/0000.parquet: downloading bytes: 0% 0.00/2.61M [00:00<?, ?B/s]
Itihasa/test/0000.parquet: downloading bytes: 8% 207k/2.61M [00:01<00:14, 170kB/s]
Itihasa/test/0000.parquet: downloading bytes: 89% 2.34M/2.61M [00:01<00:00, 2.39MB/s, 20.1kB/s ]
Itihasa/test/0000.parquet: downloading bytes: 100% 2.61M/2.61M [00:01<00:00, 1.82MB/s, 249kB/s ]
Itihasa/test/0000.parquet: reconstructing file: 100% 2.61M/2.61M [00:01<00:00, 1.82MB/s, 251kB/s ]
Generating train split: 75162 examples [00:00, 512347.91 examples/s]
Generating validation split: 6149 examples [00:00, 493678.94 examples/s]
Generating test split: 11722 examples [00:00, 562323.51 examples/s]
Loaded 6000 pairs from itihasa (revision fallback).
train=15030 val=835 test=864
Written to data/
```

```
import json
```

```
def peek(path, n=3):
    print(f"--- {path} ---")
    with open(path, encoding="utf-8") as f:
        for i, line in enumerate(f):
            if i >= n:
                break
            print(json.loads(line))
```

```
peek("data/train.jsonl")
peek("data/test.jsonl")
```

```
--- data/train.jsonl ---
{'instruction': 'Render this English sentence in Sanskrit.', 'input': 'And on that day no Brāhmaṇa ever felt tired, or hungry'}
{'instruction': 'Translate the following Sanskrit text into English.', 'input': 'त्वं राजा भरत भव स्वयं नराणां वन्यानामहमपि राजराजमृगाणां'}
{'instruction': 'Translate the following English text into Sanskrit.', 'input': 'Having placed his mothers in Ayodhyā, Bharata'}
--- data/test.jsonl ---
{'instruction': 'Provide a Sanskrit translation of this English sentence.', 'input': 'O Śatrughna, arise! Why sleepest you? Br'}
{'instruction': 'Translate the following English text into Sanskrit.', 'input': 'And presenting him with many a charming fount'}
{'instruction': 'Translate the following English text into Sanskrit.', 'input': 'And tell me, O you cognizant of morality, how
```

4. QLoRA Fine-Tuning

4-bit quantized base model + LoRA adapters on attention and MLP projection layers, trained with TRL's `SFTTrainer`. This keeps memory comfortably within a T4's 16GB, and keeps training time within the assignment's suggested 3–6 hour budget (our default config trains in well under that).

Key hyperparameters and the reasoning behind them are documented in REPORT.md section 4/5 (hardware constraints and optimizations).

```
!nvidia-smi --query-gpu=name --format=csv,noheader
```

```
Tesla T4
```

```
!python scripts/train.py \
  --base_model "$BASE_MODEL" \
  --train_file data/train.jsonl \
```

```
--val_file data/val.jsonl \
--output_dir outputs/lora-sanskrit \
--epochs 2 \
--per_device_batch_size 4 \
--grad_accum 4 \
--max_seq_len 512
```

Loading tokenizer/model: meta-llama/Llama-3.2-1B-Instruct

```
model.safetensors: downloading bytes: 7% 182M/2.47G [00:01<00:11, 205MB/s, 13.9MB/s ]
model.safetensors: downloading bytes: 10% 259M/2.47G [00:02<00:11, 193MB/s, 21.8MB/s ]
model.safetensors: downloading bytes: 23% 571M/2.47G [00:03<00:07, 267MB/s, 47.9MB/s ]
model.safetensors: downloading bytes: 55% 1.35G/2.47G [00:05<00:03, 338MB/s, 100MB/s ]
model.safetensors: downloading bytes: 80% 1.97G/2.47G [00:08<00:02, 211MB/s, 131MB/s ]
model.safetensors: downloading bytes: 87% 2.14G/2.47G [00:09<00:02, 118MB/s, 129MB/s ]
model.safetensors: reconstructing file: 73% 1.81G/2.47G [00:12<00:03, 165MB/s, 77.1MB/s ]
model.safetensors: downloading bytes: 100% 2.14G/2.14G [00:16<00:00, 126MB/s, 128MB/s ]
model.safetensors: reconstructing file: 100% 2.47G/2.47G [00:16<00:00, 146MB/s, 131MB/s ]
Loading weights: 100% 146/146 [00:01<00:00, 140.41it/s]
generation_config.json: 100% 189/189 [00:00<00:00, 733kB/s]
Loading datasets ...
Generating train split: 15030 examples [00:00, 358234.91 examples/s]
Generating train split: 835 examples [00:00, 218426.08 examples/s]
Map: 100% 500/500 [00:00<00:00, 5354.33 examples/s]
Map: 100% 835/835 [00:00<00:00, 7697.80 examples/s]
Adding EOS to train dataset: 100% 500/500 [00:00<00:00, 10291.15 examples/s]
Tokenizing train dataset: 100% 500/500 [00:00<00:00, 1532.37 examples/s]
Building labels for train dataset: 100% 500/500 [00:00<00:00, 4198.76 examples/s]
Truncating train dataset: 100% 500/500 [00:00<00:00, 4001.45 examples/s]
Dropping fully masked examples from train dataset: 100% 500/500 [00:00<00:00, 7094.63 examples/s]
Adding EOS to eval dataset: 100% 835/835 [00:00<00:00, 16574.28 examples/s]
Tokenizing eval dataset: 100% 835/835 [00:00<00:00, 1612.26 examples/s]
Building labels for eval dataset: 100% 835/835 [00:00<00:00, 4245.94 examples/s]
Truncating eval dataset: 100% 835/835 [00:00<00:00, 3902.91 examples/s]
Dropping fully masked examples from eval dataset: 100% 835/835 [00:00<00:00, 6951.17 examples/s]
Starting training ...
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model cc
{'loss': '4.015', 'grad_norm': '1.883', 'learning_rate': '6.786e-05', 'entropy': '3.258', 'num_tokens': '4.863e+04', 'mean_tc
{'loss': '2.745', 'grad_norm': '1.383', 'learning_rate': '0.0001393', 'entropy': '2.842', 'num_tokens': '9.571e+04', 'mean_tc
{'loss': '2.4', 'grad_norm': '1.445', 'learning_rate': '0.0001383', 'entropy': '2.472', 'num_tokens': '1.437e+05', 'mean_tke
100% 64/64 [13:57<00:00, 10.56s/it]
0% 0/209 [00:00<?, ?it/s]
1% 2/209 [00:00<01:25, 2.42it/s]
1% 3/209 [00:01<01:54, 1.79it/s]
2% 4/209 [00:02<02:31, 1.35it/s]
2% 5/209 [00:03<02:32, 1.34it/s]
3% 6/209 [00:04<02:32, 1.33it/s]
3% 7/209 [00:05<02:51, 1.18it/s]
4% 8/209 [00:06<02:54, 1.15it/s]
4% 9/209 [00:07<03:12, 1.04it/s]
5% 10/209 [00:08<03:12, 1.03it/s]
5% 11/209 [00:09<02:57, 1.11it/s]
6% 12/209 [00:10<03:10, 1.03it/s]
6% 13/209 [00:10<03:01, 1.08it/s]
7% 14/209 [00:11<02:51, 1.14it/s]
7% 15/209 [00:12<03:07, 1.04it/s]
8% 16/209 [00:13<03:03, 1.05it/s]
8% 17/209 [00:15<03:27, 1.08s/it]
9% 18/209 [00:16<03:27, 1.09s/it]
9% 19/209 [00:17<03:39, 1.15s/it]
10% 20/209 [00:18<03:14, 1.03s/it]
10% 21/209 [00:20<04:29, 1.43s/it]
11% 22/209 [00:21<03:48, 1.22s/it]
11% 23/209 [00:22<03:34, 1.15s/it]
11% 24/209 [00:23<03:45, 1.06s/it]
```

If you hit an OOM on a T4: lower `--per_device_batch_size` to 2 and raise `--grad_accum` to 8 (same effective batch size, less peak memory), or lower `--max_seq_len` to 384. This tradeoff is discussed in REPORT.md.

For a quick smoke test before committing to a full run, add `--max_train_samples 500` to finish a sanity-check pass in a few minutes.

5. Inference: Base Model vs Fine-Tuned

We run both the untouched base model and our fine-tuned adapter over the same held-out test set, so we can directly compare before/after outputs.

```
# Base model predictions (no adapter) – this is the "before" baseline
!python scripts/infer.py \
  --base_model "$BASE_MODEL" \
  --test_file data/test.jsonl \
  --out_file eval/predictions_base.jsonl \
  --max_new_tokens 150
```

```
Loading weights: 100% 146/146 [00:00<00:00, 159.23it/s]
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clean_up_tokenization post-
10/100 done
20/100 done
30/100 done
40/100 done
50/100 done
60/100 done
70/100 done
80/100 done
90/100 done
100/100 done
Wrote 100 predictions to eval/predictions_base.jsonl
```

```
# Fine-tuned model predictions – this is "after"
!python scripts/infer.py \
  --base_model "$BASE_MODEL" \
  --adapter outputs/lora-sanskrit \
  --test_file data/test.jsonl \
  --out_file eval/predictions.jsonl \
  --max_new_tokens 150
```

```
Loading weights: 100% 146/146 [00:00<00:00, 160.51it/s]
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clean_up_tokenization post-
10/100 done
20/100 done
30/100 done
40/100 done
50/100 done
60/100 done
70/100 done
80/100 done
90/100 done
100/100 done
Wrote 100 predictions to eval/predictions.jsonl
```

```
# Quick interactive sanity check on a single example
!python scripts/infer.py \
  --base_model "$BASE_MODEL" \
  --adapter outputs/lora-sanskrit \
  --instruction "Translate the following Sanskrit text into English." \
  --input "वसुधैव कुटुम्बकम्।"
```

```
Loading weights: 100% 146/146 [00:00<00:00, 158.16it/s]
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clean_up_tokenization post-

=== PREDICTION ===
And there was a son of Vasudeva.
```

6. Evaluation

We compute BLEU and chrF++ per task type, generate a before-vs-after comparison table, and run a lightweight heuristic pass to flag likely failure cases (empty outputs, verbatim-echo, rambling/hallucinated length, degenerate repetition) for manual review.

chrF++ is emphasized over BLEU for Sanskrit because it's character-level and more forgiving of compounding/sandhi variation than word-level n-gram matching — see REPORT.md section 6 for the full discussion of why n-gram metrics are a limited signal here, especially for the non-translation task types (explanation/QA/summarization), which we evaluate mainly qualitatively.

```
!python eval/evaluate.py \
  --base_predictions eval/predictions_base.jsonl \
```

```

wrote report to eval/report.md
{
  "en2sa": {
    "n": 45,
    "bleu": 0.08,
    "chrfr++": 10.21
  },
  "explain": {
    "n": 15,
    "bleu": 5.0,
    "chrfr++": 21.35
  },
  "qa": {
    "n": 10,
    "bleu": 0.98,
    "chrfr++": 12.75
  },
  "sa2en": {
    "n": 24,
    "bleu": 1.15,
    "chrfr++": 15.6
  },
  "summary": {
    "n": 6,
    "bleu": 0.84,
    "chrfr++": 15.2
  }
}

```

```
- **Flags:** repetitive_degeneration | **Task:** en2sa
- Input: `And presenting him with many a charming fountain, trees will delight Rāma at the tops of mountains.* Where Rama i
- Reference: `पादयुः पर्वताग्रेषु रमिष्यन्ति राघवम्। यत्र रामो भूय नात्र नास्ति तत्र प्राभवः॥ स हि शूरो महाबाहुः पुन्रो दशरथस्य च। पुर'
- Prediction: `ततो रामस्य ते विश्राव्यं त्वं प्रियं त्वं प्रियम्। दशरथस्य महामने ते न कालं न कालं न कालम्॥ रामस्य ते विश्राव्यं ते वि'
- **Flags:** much_longer_than_reference, repetitive_degeneration | **Task:** en2sa
- Input: `And tell me, O you cognizant of morality, how far is it to that high-souled righteous one's asyln, and by what wa
- Reference: `आश्रमं तस्य धर्मज्ञं धार्मिकस्य महात्मनः। आचक्ष्व कतमो मार्गः कियानिति च शंस मे॥`
- Prediction: `तस्माद् विद्या ते मुनिः कुशलस्ते त्वं कुशलस्ते त्वं। त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्वं त्`
- **Flags:** much_longer_than_reference, repetitive_degeneration | **Task:** sa2en
- Input: `सात्वत्प्रि प्रविश वा स्वयं वा विश दण्डकान्। रज्जु बद्ध्वाथवा कण्ठे नहि तेऽन्यत् परायणम्॥`
- Reference: `Do you enter fire, or dive into Dandaka, or wound a cord round your neck: other desirable way there is none f
- Prediction: `O mighty one, do you go to the forest, or stay with me, or do you go to the forest, or do you go to the fore
- **Flags:** repetitive_degeneration | **Task:** sa2en
- Input: `हत्वा तु तं भीमबलं विराधं राक्षसं वने। ततः सीतां परिष्पृज्य समाश्वास्य च वीर्यवान्॥ अन्नवीढ् भ्रातरं रामो लक्ष्मणं दीप्तत'
- Reference: `Having in the forest slain the Rākṣasa Virādha possessed of tremendous strength, Rāma endowed with prowess, em
- Prediction: `O mighty one, I have come to Sītā, and I have been banished from the forest. I have been deprived of my brot
- **Flags:** repetitive_degeneration | **Task:** qa
```

```

- Input: `एष राम शिवः पन्था यत्रैते पुष्पिता द्रुमाः। प्रतीची दिशमाश्रित्य प्रकाशन्ते मनोरमाः॥ जम्बूप्रियालपनसा न्यग्रोधपक्षतिन्दु`
- Reference: `This is the way, O Rāma, leading to the mount Rsyamūka, where stand, beautifying the West, the Jambū (a) Priy
- Prediction: `O Rāma, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Sitā, O Si
- **Flags:** much longer than reference, repetitive degeneration | **Task:** sa2en
- Input: `न द्वेष्ट विद्यते तस्य स तु द्वेष्टि न कंचन। स तु सर्वेषु भूतेषु पितामह इवापरः॥`
- Reference: `He has no hater; nor does he hate any one. And in relation to all creatures he was like another great-father.
- Prediction: `And thereupon, O Rāma, do you not see him, and do you not see him, and do you not see him, and do you not se
- **Flags:** much longer than reference, repetitive degeneration | **Task:** sa2en
- Input: `तौ लोहितस्य प्रियदर्शनस्य सदोचितवुत्तमचन्दनस्य। वृत्तौ स्तनौ शोणितपङ्कदिग्धौ नूनं प्रियाया मम नाभिपातः॥`
- Reference: `For certain my dear wife's breast round and sprinkled as it was with red sandal paste, was bathed in blood (w
- Prediction: `O my beloved, I have been told of the excellent counsel of the lord of the earth, and the excellent qualitie
- **Flags:** repetitive degeneration | **Task:** en2sa
- Input: `Hearing those words of the intelligent prince, that righteous (ascetic) brought a bull* as well as Arghya and wat
- Reference: `तस्य तद् वचनं श्रुत्वा राजपुत्रस्य धीमतः। उपानयत धर्मात्मा गामर्थ्यमुदकं ततः॥`

```

Per-example ranking (sentence-level chrF++). `evaluate.py` reports corpus-level metrics and a heuristic failure flagger, but doesn't rank individual predictions. The cell below scores every fine-tuned prediction with `sacrebleu.sentence_chrf` and sorts, so the "best"/"worst" examples used in the report are selected automatically and reproducibly, rather than by manually reading through `eval/report.md` (see REPORT.md section 6/7 for the resulting examples and discussion).

```

import sys
sys.path.insert(0, "eval") # so we can import load_jsonl from evaluate.py
from evaluate import load_jsonl
import sacrebleu

def score_and_rank(predictions_file, task_type_filter=None, top_n=5, bottom_n=5, max_chars=200):
    """Score every prediction individually (sentence-level chrF++), rank them,
    and print the best- and worst-scoring examples. This is how you find
    genuine successes/failures instead of eyeballing and cherry-picking.

    Uses the same jsonl schema infer.py writes: instruction, input, reference,
    prediction, task_type.
    """
    rows = load_jsonl(predictions_file)
    if task_type_filter:
        rows = [r for r in rows if r.get("task_type") == task_type_filter]
    if not rows:
        print(f"No rows found (task_type_filter={task_type_filter!r}).")
        return []

    def trunc(s, n=max_chars):
        s = s.replace("\n", " ").strip()
        return s if len(s) <= n else s[:n].rstrip() + "..."

    for r in rows:
        pred = r.get("prediction", "").strip()
        ref = r.get("reference", "").strip()
        # sentence_chrf errors on an empty hypothesis/reference pair -> score 0 instead
        if not pred or not ref:
            r["_chrf"] = 0.0
        else:
            r["_chrf"] = sacrebleu.sentence_chrf(pred, [ref], word_order=2).score

    ranked = sorted(rows, key=lambda r: r["_chrf"], reverse=True)

    print(f"=== TOP {top_n} (highest chrF++) – candidates for 'successful cases' in REPORT.md ===\n")
    for r in ranked[:top_n]:
        print(f"chrF++={r['_chrf']:.1f} task={r['task_type']}")
        print(f"Input: {trunc(r.get('input', ''))}")
        print(f"Reference: {trunc(r.get('reference'))}")
        print(f"Prediction: {trunc(r.get('prediction'))}\n")

    print(f"=== BOTTOM {bottom_n} (lowest chrF++) – candidates for 'failure analysis' in REPORT.md ===\n")
    for r in ranked[-bottom_n:]:
        print(f"chrF++={r['_chrf']:.1f} task={r['task_type']}")
        print(f"Input: {trunc(r.get('input', ''))}")
        print(f"Reference: {trunc(r.get('reference'))}")
        print(f"Prediction: {trunc(r.get('prediction'))}\n")

```

```
===== task_type = sa2en =====
=== TOP 3 (highest chrF++) - candidates for 'successful cases' in REPORT.md ===
```

```
import shutil
shutil.make_archive("lora-sanskrit-adapter", "zip", "outputs/lora-sanskrit")
print("Zipped adapter -> lora-sanskrit-adapter.zip")

from google.colab import files
files.download("lora-sanskrit-adapter.zip")
```

Zipped adapter -> lora-sanskrit-adapter.zip

```
# Optional: push adapter to the Hugging Face Hub instead of downloading a zip
# from huggingface_hub import HfApi
# api = HfApi()
# api.create_repo("your-username/sanskrit-llama-lora", exist_ok=True)
# api.upload_folder(folder_path="outputs/lora-sanskrit", repo_id="your-username/sanskrit-llama-lora")
```

Next Steps / What We'd Improve With More Time

See [REPORT.md](#) section 9 for the full list. Highlights:

- Tokenizer adaptation (extend vocab with Devanagari-aware merges) given the fragmentation ratio measured in section 2 above
- Larger, more diverse Sanskrit sources (GRETIL, Digital Corpus of Sanskrit, AI4Bharat) beyond itihasa's epic-poetry register, to generalize past Ramayana/Mahabharata style text
- Human evaluation (native Sanskrit speaker) rather than only automatic metrics, especially for the explanation/QA tasks
- LoRA-vs-full-fine-tune and quantization ablations (bonus items)